3 - *PERF* INTERFACE

3.1 Introdução

O `perf` é uma interface de linha de comando para o subsistema `perf_events` do Linux. Seu objetivo é permitir que aplicações, processos, CPUs ou o sistema inteiro sejam observados por meio de eventos de desempenho. Esses eventos podem vir de diferentes fontes:

- contadores de hardware da CPU (PMU);
- contadores de software mantidos pelo kernel;
- eventos de cache e TLB;
- tracepoints do kernel;
- métricas derivadas da microarquitetura;
- amostragem para localizar funções e trechos de código quentes;
- eventos de energia, quando expostos pela plataforma.

A PMU contém registradores especiais capazes de contar ocorrências de eventos enquanto um programa executa. Exemplos clássicos são:

- ciclos de CPU;
- instruções aposentadas;
- desvios executados;
- erros de predição de desvio;
- referências e faltas de cache;
- eventos de TLB;
- micro-operações e eventos específicos da microarquitetura.

Uma visão simplificada é:

Aplicação
↓
`perf`
↓
`perf_events` no kernel Linux
↓
PMU do processador
↓
cycles, instructions, branches, cache misses, etc.

A vantagem do `perf` é que o kernel fornece uma camada de abstração sobre diferenças entre processadores. Entretanto, a lista exata de eventos disponíveis continua dependendo da arquitetura, do modelo da CPU, do kernel e dos drivers PMU presentes no sistema.

3.2 Instalação no Debian

No Debian, a ferramenta é fornecida pelo pacote `linux-perf`. A instalação básica é:

```
sudo apt update
sudo apt install linux-perf
```

Depois da instalação, verifique:

```
perf --version
```

O pacote `linux-perf` faz parte do conjunto de ferramentas de análise de desempenho do kernel disponibilizado pelo Debian.

3.2.1 Verificando permissões

Alguns eventos de desempenho podem ser restringidos pelo kernel. Um parâmetro importante é:

```
cat /proc/sys/kernel/perf_event_paranoid
```

Valores mais restritivos podem impedir que usuários comuns leiam determinados contadores. Antes de modificar esse parâmetro, recomenda-se testar o comando desejado. Se o `perf` funcionar normalmente, não há motivo para relaxar a política de segurança. Verificar as orientações de configuração do `kernel` descritas no Capítulo 2 Sessão 2.1.

3.3 Primeiros comandos

3.3.1 Listando eventos disponíveis

A lista completa depende da máquina:

```
perf list
```

É possível filtrar por classes:

```
perf list hw
perf list sw
perf list cache
perf list pmu
perf list metric
perf list metricgroup
perf list tracepoint
```

O comando `perf list` deve ser considerado a fonte de verdade para descobrir quais nomes de eventos são realmente aceitos no sistema em uso.

3.3.2 Medição básica com `perf stat`

O uso mais simples consiste em executar uma aplicação sob o comando `perf stat`:

```
perf stat ./programa
```

Para o resolvedor de Gauss usado como exemplo:

```
perf stat ./gauss.exec 42 1000 1E-10
```

O `perf stat` executa o programa e acumula estatísticas dos eventos selecionados.

3.4 Interpretando a saída básica do `perf stat`

Considere uma saída real obtida para um resolvedor sequencial de Gauss em um Intel Core i3-1215U:

```
Performance counter stats for './gauss.exec 42 1000 1E-10':

     181.28 msec task-clock            # 0.989 CPUs utilized
          4      context-switches      # 22.065 /sec
          1      cpu-migrations        # 5.516 /sec
```

```
        982      page-faults           # 5.417 K/sec
195,047,810      cpu_atom/cycles/      # 1.076 GHz       (0.94%)
663,118,440      cpu_core/cycles/      # 3.658 GHz      (94.70%)
154,556,985      cpu_atom/instructions/# 0.79 insn/cycle  (3.13%)
2,982,655,890    cpu_core/instructions/# 4.50 insn/cycle (94.70%)
26,407,510       cpu_atom/branches/    # 145.670 M/sec    (5.24%)
371,181,951      cpu_core/branches/    # 2.048 G/sec     (94.70%)
1,001,131        cpu_atom/branch-misses/# 3.79%           (5.30%)
1,571,719        cpu_core/branch-misses/# 0.42%           (94.70%)

TopdownL1 (cpu_core)
  21.7 % tma_backend_bound
   6.4 % tma_bad_speculation
   5.4 % tma_frontend_bound
  66.5 % tma_retiring

TopdownL1 (cpu_atom)
   5.4 % tma_bad_speculation
  19.6 % tma_retiring
  61.0 % tma_backend_bound
  14.0 % tma_frontend_bound

0.183207990 seconds time elapsed
0.174962000 seconds user
0.007952000 seconds sys
```

As subseções seguintes explicam cada grupo de valores.

3.4.1 `task-clock`

A linha

```
181.28 msec task-clock   # 0.989 CPUs utilized
```

indica aproximadamente o tempo total de CPU consumido pela tarefa. O tempo de relógio da execução foi:

```
0.183207990 seconds time elapsed
```

Logo,

$$\frac{0.18128}{0.183208} \approx 0.989.$$

O valor `0.989 CPUs utilized` é compatível com uma aplicação sequencial que ocupa praticamente uma CPU lógica durante toda a execução.

Em uma aplicação paralela, o valor pode ser maior que 1. Por exemplo, se quatro threads estiverem efetivamente executando em quatro CPUs durante quase todo o intervalo, a utilização pode se aproximar de 4.

3.4.2 Trocas de contexto

```
4 context-switches
```

Uma troca de contexto ocorre quando o scheduler deixa de executar a tarefa naquela CPU para executar outra entidade. Um pequeno número de context switches normalmente não representa problema para um benchmark curto.

Uma quantidade elevada pode indicar interferência do sistema operacional, competição com outros processos, bloqueios ou comportamento de sincronização relevante.

3.4.3 Migrações de CPU

```
1 cpu-migrations
```

Uma migração ocorre quando uma thread que estava em uma CPU lógica passa a executar em outra. Para benchmarks controlados isso pode ser indesejável porque diferentes CPUs podem apresentar:

- frequências distintas;

- diferentes estados de cache;
- diferentes domínios NUMA;
- diferentes microarquiteturas, em CPUs híbridas.

Por isso, experimentos de HPC frequentemente fixam afinidade com `taskset` ou outra ferramenta de pinagem.

3.4.4 Page faults

```
982 page-faults
```

Um page fault não implica necessariamente acesso ao disco. Sistemas operacionais modernos usam paginação sob demanda: a memória virtual pode ser reservada antes de a página física ser efetivamente associada ao processo.

É útil separar faltas menores e maiores:

```
perf stat -e minor-faults,major-faults \
    ./gauss.exec 42 1000 1E-10
```

Faltas maiores são mais preocupantes porque podem exigir acesso a armazenamento secundário.

3.5 Cycles, Instructions, IPC e CPI

Dois dos eventos mais importantes em análise de desempenho são:

- `cycles`: ciclos de CPU;
- `instructions`: instruções aposentadas.

No P-core do exemplo:

```
663,118,440   cpu_core/cycles/
2,982,655,890 cpu_core/instructions/
```

O IPC (*Instructions Per Cycle*) é:

$$IPC = \frac{\text{instructions}}{\text{cycles}}.$$

Portanto,

$$IPC \approx \frac{2.982.655.890}{663.118.440} \approx 4.50.$$

O `perf` já mostra essa razão:

```
# 4.50 insn per cycle
```

O CPI (*Cycles Per Instruction*) é o inverso:

$$CPI = \frac{\text{cycles}}{\text{instructions}} = \frac{1}{IPC}.$$

Assim,

$$CPI \approx \frac{1}{4.50} \approx 0.222.$$

3.5.1 Por que IPC pode ser maior que 1?

Processadores modernos são supersescalares. Eles podem buscar, decodificar, emitir, executar e aposentar várias instruções por ciclo, desde que haja paralelismo suficiente e recursos internos disponíveis.

Consequentemente, IPC maior que 1 não é anormal. Também não se deve comparar IPC isoladamente entre programas diferentes sem considerar o tipo de instrução executado. Uma versão otimizada pode executar menos instruções, mas cada instrução pode realizar mais trabalho, especialmente quando há vetorização SIMD.

3.6 Branches e branch misses

No exemplo:

```
371,181,951 cpu_core/branches/
1,571,719   cpu_core/branch-misses/
```

A taxa de erro de predição é:

$$\text{Branch Miss Rate} = \frac{\text{branch misses}}{\text{branches}} \times 100.$$

Logo,

$$\frac{1.571.719}{371.181.951} \times 100 \approx 0.42\%.$$

Uma taxa pequena sugere que a predição de desvios não é o gargalo dominante. Entretanto, a interpretação deve considerar o custo de cada erro: um único branch mal previsto pode fazer várias micro-operações especulativas serem descartadas.

3.7 CPUs híbridas: `cpu_core` e `cpu_atom`

Processadores Intel híbridos, como os Alder Lake, possuem dois tipos de núcleo:

P-core *Performance Core*: otimizado para alto desempenho por thread;

E-core *Efficient Core*: otimizado para eficiência energética e densidade de throughput.

No Linux, o `perf` pode expor as PMUs dessas duas microarquiteturas separadamente:

Nome no `perf`	Núcleo	Papel geral
`cpu_core`	P-core	maior desempenho por thread
`cpu_atom`	E-core	maior eficiência e densidade

O nome `cpu_atom` é um identificador histórico/técnico usado para a PMU da linhagem dos E-cores. Ele não significa que o processador inteiro seja um Intel Atom.

3.7.1 Por que as PMUs são separadas?

P-cores e E-cores não são microarquiteturalmente idênticos. Seus contadores programáveis, eventos, codificações e capacidades podem diferir. Por isso, o kernel precisa representar as PMUs como dispositivos distintos.

É possível verificar as PMUs disponíveis em:

```
ls /sys/bus/event_source/devices/
```

E, em sistemas híbridos, descobrir quais CPUs lógicas pertencem a cada tipo:

```
cat /sys/bus/event_source/devices/cpu_core/cpus
cat /sys/bus/event_source/devices/cpu_atom/cpus
```

3.7.2 Atenção aos percentuais entre parênteses

Uma saída pode apresentar:

```
663,118,440 cpu_core/cycles/       (94.70%)
195,047,810 cpu_atom/cycles/        (0.94%)
```

Esses percentuais não devem ser lidos simplesmente como “94,70% do programa rodou em P-core” e “0,94% em E-core”. Eles estão relacionados ao tempo em que o evento esteve efetivamente ativo/contado em comparação com o tempo em que esteve habilitado, podendo refletir multiplexação e escalonamento dos contadores.

Como uma PMU possui quantidade limitada de contadores físicos, ao solicitar muitos eventos o kernel pode alternar entre eles. O `perf` então escala os valores para estimar a contagem correspondente ao período completo.

Quanto mais baixo esse percentual, maior deve ser o cuidado na interpretação. Comparações entre razões derivadas de eventos que tiveram tempos ativos muito diferentes também podem se tornar pouco confiáveis.

3.8 Afinidade: evitando migração durante benchmarks

Para experimentos reprodutíveis, é recomendável fixar o processo em uma CPU lógica específica.

Primeiro descubra os conjuntos de CPUs:

```
cat /sys/bus/event_source/devices/cpu_core/cpus
cat /sys/bus/event_source/devices/cpu_atom/cpus
```

Supondo que a CPU lógica 0 seja um P-core:

```
taskset -c 0 ./gauss.exec 42 1000 1E-10
```

Com `perf`:

```
perf stat \
    -e cpu_core/cycles/ \
    -e cpu_core/instructions/ \
    -e cpu_core/branches/ \
    -e cpu_core/branch-misses/ \
    taskset -c 0 ./gauss.exec 42 1000 1E-10
```

O mesmo princípio pode ser usado para medir um E-core, selecionando uma CPU pertencente a `cpu_atom`.

3.9 Repetição e variabilidade

Uma única execução não é suficiente para caracterizar desempenho com rigor. O `perf stat` pode repetir automaticamente o benchmark:

```
perf stat -r 10 ./gauss.exec 42 1000 1E-10
```

O uso de múltiplas repetições ajuda a observar variabilidade causada por:

- scheduler;
- interrupções;
- frequência dinâmica;
- temperatura e throttling;
- outros processos;
- estado dos caches;
- alocação de memória.

Para estudos acadêmicos, deve-se registrar não apenas a média, mas também alguma medida de dispersão, como desvio padrão, coeficiente de variação ou intervalo de confiança.

3.10 Eventos de cache

A hierarquia de memória é frequentemente um dos principais gargalos em HPC. O `perf` pode expor eventos genéricos e eventos específicos da CPU.

Uma medição básica é:

```
perf stat -e cache-references,cache-misses \
    ./gauss.exec 42 1000 1E-10
```

Quando disponíveis, eventos mais detalhados podem incluir:

```
perf stat -e \
    L1-dcache-loads,L1-dcache-load-misses \
    ./gauss.exec 42 1000 1E-10
```

E para o último nível de cache:

```
perf stat -e \
    LLC-loads,LLC-load-misses \
    ./gauss.exec 42 1000 1E-10
```

A taxa de misses pode ser calculada por:

$$\text{Miss Rate} = \frac{\text{misses}}{\text{accesses}} \times 100.$$

O nome exato e a disponibilidade dos eventos devem sempre ser verificados com `perf list`.

3.11 Eventos de TLB

A TLB (*Translation Lookaside Buffer*) armazena traduções recentes entre endereços virtuais e físicos. Uma falta na TLB pode exigir um *page-table walk*, adicionando latência.

Quando disponíveis:

```
perf stat -e dTLB-loads,dTLB-load-misses \
    ./gauss.exec 42 1000 1E-10
```

A cadeia conceitual é:

$$\text{endereço virtual} \rightarrow \text{TLB} \rightarrow \text{endereço físico} \rightarrow \text{cache} \rightarrow \text{memória}$$

Matrizes grandes podem aumentar pressão sobre TLB e caches, especialmente quando o padrão de acesso possui baixa localidade.

3.12 Atalho para estatísticas detalhadas: opção `-d`

O `perf stat` possui níveis de detalhamento predefinidos:

```
perf stat -d ./programa
perf stat -d -d ./programa
perf stat -d -d -d ./programa
```

Esses níveis acrescentam conjuntos adicionais de eventos, sujeitos à disponibilidade na plataforma. É uma forma prática de explorar rapidamente cache, TLB e outros contadores antes de selecionar eventos manualmente.

Para o exemplo:

```
perf stat -d -d ./gauss.exec 42 1000 1E-10
```

3.13 Análise Top-down da microarquitetura

A metodologia Top-Down Microarchitecture Analysis divide os slots da pipeline em categorias que ajudam a identificar o gargalo dominante. No nível mais alto, as quatro categorias são:

Retiring trabalho que efetivamente chega à aposentadoria;

Frontend Bound o frontend não entrega micro-operações em ritmo suficiente;

Backend Bound o backend não consegue processar as operações em ritmo suficiente;

Bad Speculation trabalho especulativo que precisou ser descartado.

Uma forma de solicitar a análise é:

```
perf stat --topdown ./gauss.exec 42 1000 1E-10
```

No P-core do exemplo:

```
21.7 % tma_backend_bound
 6.4 % tma_bad_speculation
 5.4 % tma_frontend_bound
66.5 % tma_retiring
```

Essas categorias somam aproximadamente 100% porque classificam os slots da pipeline no primeiro nível do modelo.

3.13.1 Retiring

No exemplo:

$$Retiring = 66.5\%.$$

Uma parcela grande de Retiring indica que muitos slots estão associados a operações que chegaram à aposentadoria. É normalmente uma condição desejável, mas não significa automaticamente que todo trabalho aposentado seja algoritmicamente útil. Operações adicionais geradas pelo compilador, microcódigo e outros efeitos também podem contribuir.

3.13.2 Frontend Bound

No exemplo:

$$FrontendBound = 5.4\%.$$

O frontend é responsável por buscar e decodificar instruções e alimentar o restante da pipeline. Um valor baixo sugere que a alimentação do backend não é o problema dominante.

Possíveis causas de frontend bound incluem:

- misses de cache de instruções;
- problemas de decodificação;
- código muito grande ou com fluxo de controle desfavorável;
- baixa disponibilidade de micro-operações no frontend.

3.13.3 Bad Speculation

No exemplo:

$$BadSpeculation = 6.4\%.$$

Essa categoria representa slots desperdiçados por especulação incorreta. Branch misprediction é uma causa importante, mas a métrica Top-down não deve ser confundida com a taxa `branch-misses/branches`: os denominadores são diferentes.

3.13.4 Backend Bound

No exemplo:

$$BackendBound = 21.7\%.$$

O backend pode ser limitado por duas famílias de causas:

$$BackendBound \rightarrow \begin{cases} MemoryBound, \\ CoreBound. \end{cases}$$

Memory Bound pode envolver cache misses, latência de DRAM, largura de banda de memória ou TLB. Já **Core Bound** envolve limitações internas das unidades de execução, dependências de dados, pressão sobre portas, latência de operações e outras restrições do núcleo.

Para um Gauss, investigar Backend Bound é particularmente interessante porque o algoritmo combina operações de ponto flutuante com acessos intensivos a matrizes.

3.14 Métricas e grupos de métricas

O `perf` pode oferecer métricas derivadas e grupos preparados para determinada microarquitetura:

```
perf list metric
perf list metricgroup
```

Uma métrica pode ser solicitada com `-M`, por exemplo:

```
perf stat -M NOME_DA_METRICA ./programa
```

Os nomes disponíveis variam entre processadores. Em análises Intel modernas, vários grupos podem corresponder a níveis adicionais da metodologia Top-down.

3.15 Ponto flutuante, SIMD e vetorização

O `perf` também pode acessar eventos específicos da PMU relacionados a ponto flutuante e vetorização, quando a arquitetura e a base de eventos do sistema os disponibilizam.

Uma forma segura de descobrir esses eventos é:

```
perf list | grep -i fp
perf list | grep -Ei 'arith|vector|simd|avx'
```

Diferentemente de ferramentas como LIKWID, que fornecem grupos prontos como `FLOPS_DP`, no `perf` frequentemente se trabalha mais próximo dos eventos individuais da PMU.

Uma aplicação vetorizada pode executar uma instrução SIMD que atua simultaneamente sobre vários elementos. Por exemplo, uma instrução de 256 bits pode conter quatro valores `double` de 64 bits:

$$\frac{256}{64} = 4.$$

Portanto, a contagem de instruções FP não deve ser automaticamente interpretada como a contagem de FLOPs. É necessário conhecer a semântica do evento e a largura vetorial.

3.16 `perf record`, `perf report` e profiling por função

`perf stat` responde essencialmente à pergunta:

> Quantas vezes determinados eventos aconteceram durante a execução?

Para localizar onde o tempo é gasto, usa-se amostragem:

```
perf record ./gauss.exec 42 1000 1E-10
```

O comando grava um arquivo `perf.data`. Depois:

```
perf report
```

O relatório mostra funções associadas às amostras coletadas e ajuda a identificar *hotspots*.

Para relacionar amostras com assembly e, quando os símbolos/informações de depuração estão disponíveis, com linhas do código:

```
perf annotate
```

Para melhorar a qualidade da atribuição de amostras ao código, é frequentemente útil compilar com símbolos de depuração:

```
g++ -O3 -g -march=native gauss.cpp -o gauss.exec
```

O uso de `-g` não implica necessariamente desativar otimizações; ele adiciona informações de depuração que ajudam ferramentas de profiling.

3.17 Eventos de software e tracepoints

Além da PMU, o `perf` trabalha com eventos de software mantidos pelo kernel e tracepoints.

Exemplos de eventos de software incluem:

- `task-clock`;
- `context-switches`;
- `cpu-migrations`;
- `page-faults`;
- `minor-faults`;
- `major-faults`.

Tracepoints permitem observar eventos internos do kernel. A lista pode ser visualizada com:

```
perf list tracepoint
```

Esse mecanismo estende o uso do `perf` para além do profiling de CPU, permitindo investigar scheduler, sistemas de arquivos, chamadas do kernel, rede e outros subsistemas.

3.18 Energia e Intel RAPL

Em plataformas compatíveis, Linux pode expor contadores de energia do Intel RAPL (*Running Average Power Limit*). O RAPL organiza o consumo em domínios ou zonas de energia.

Primeiro procure os eventos disponíveis:

```
perf list power
perf list | grep -i energy
```

Dependendo da plataforma, podem aparecer eventos como:

```
power/energy-pkg/
power/energy-cores/
power/energy-ram/
power/energy-gpu/
power/energy-psys/
```

Nem todos os domínios existem em todas as CPUs.

Uma medição típica de energia do package é:

```
sudo perf stat -a \
    -e power/energy-pkg/ \
    ./gauss.exec 42 3000 1E-10
```

O resultado é energia, normalmente expressa em joules. A potência média pode ser estimada por:

$$P_{medio} = \frac{E}{t},$$

onde E é energia em joules e t é tempo em segundos.

Como

$$1\,W = 1\,J/s,$$

a razão fornece watts médios durante o intervalo.

3.18.1 O que RAPL não mede

O evento do package não deve ser confundido com o consumo total do computador. Um sistema contém outros consumidores:

- GPU discreta;
- memória e controladores não cobertos pelo domínio medido;
- SSDs e outros dispositivos de armazenamento;
- chipset;
- interfaces de rede;
- ventiladores;
- tela de notebook;
- perdas na fonte de alimentação.

Assim,

$$E_{package} \neq E_{computador\ inteiro}.$$

Para consumo total na tomada, a abordagem de referência é utilizar um medidor externo de potência/energia.

3.18.2 Energia não é potência instantânea

O framework de powercap do kernel expõe contadores acumulados de energia, como `energy_uj`. O próprio kernel observa que o RAPL não necessariamente fornece potência instantânea; esta pode ser estimada pela variação de energia em um intervalo de tempo.

3.18.3 Métricas para Green HPC

A comparação de versões pode usar energia e tempo simultaneamente. O *Energy-Delay Product* é:

$$EDP = E \times T.$$

Outra métrica é:

$$ED^2P = E \times T^2.$$

Essas métricas ajudam a evitar conclusões baseadas exclusivamente em potência ou energia. Uma versão pode ter maior potência média, mas terminar tão mais rápido que consome menos energia total.

3.19 Exemplo de conjunto de eventos para HPC

Um conjunto inicial útil para uma aplicação sequencial pode ser:

```
perf stat \
    -e cycles \
    -e instructions \
    -e branches \
    -e branch-misses \
    -e cache-references \
    -e cache-misses \
    -e context-switches \
    -e cpu-migrations \
    -e page-faults \
    ./gauss.exec 42 1000 1E-10
```

Em uma CPU híbrida, é melhor selecionar explicitamente a PMU e fixar a afinidade quando o objetivo é estudar um tipo de núcleo.

Por exemplo, para um P-core:

```
perf stat -r 10 \
    -e cpu_core/cycles/ \
    -e cpu_core/instructions/ \
    -e cpu_core/branches/ \
    -e cpu_core/branch-misses/ \
    taskset -c 0 ./gauss.exec 42 1000 1E-10
```

3.20 Proposta de fluxo de profiling para uma aplicação HPC

Um fluxo sistemático evita coletar dezenas de métricas sem uma pergunta clara.

3.20.1 Etapa 1: validar o resultado

Antes de medir desempenho, o programa deve produzir um resultado correto. No exemplo do Gauss, o erro máximo é comparado a uma tolerância.

3.20.2 Etapa 2: medir tempo e variabilidade

```
perf stat -r 10 ./gauss.exec 42 1000 1E-10
```

Pergunta: o tempo é estável?

3.20.3 Etapa 3: medir instruções e ciclos

Perguntas:

- Quantas instruções são executadas?
- Quantos ciclos são gastos?
- Qual é o IPC/CPI?

3.20.4 Etapa 4: verificar desvios

Pergunta: branch misprediction é significativo?

3.20.5 Etapa 5: usar Top-down

Pergunta: o gargalo dominante está no frontend, backend ou especulação?

3.20.6 Etapa 6: decompor Backend Bound

Se Backend Bound for relevante, investigar:

- cache L1/L2/LLC;
- TLB;
- memória/DRAM;
- dependências e unidades de execução;
- largura de banda;
- vetorização.

3.20.7 Etapa 7: localizar hotspots

```
perf record ./gauss.exec 42 1000 1E-10
perf report
```

Pergunta: quais funções realmente dominam o custo?

3.20.8 Etapa 8: otimizar e repetir

Depois de alterar compilação ou código, repetir exatamente as mesmas condições experimentais.

3.21 Comparando níveis de otimização

Um experimento didático muito útil é comparar:

```
g++ -O0 gauss.cpp -o gauss-O0.exec
g++ -O2 gauss.cpp -o gauss-O2.exec
g++ -O3 gauss.cpp -o gauss-O3.exec
g++ -O3 -march=native gauss.cpp -o gauss-native.exec
```

Para cada versão, coletar:

- tempo;
- cycles;
- instructions;
- IPC/CPI;
- branch misses;
- cache misses;
- Top-down;
- eventos FP/SIMD, quando disponíveis;
- energia, quando suportada.

A otimização pode diminuir drasticamente o número de instruções e, ao mesmo tempo, aumentar o CPI. Isso não é contradição: instruções vetoriais ou mais complexas podem realizar mais trabalho por instrução. O objetivo final é reduzir o tempo e/ou energia para uma quantidade correta de trabalho, não minimizar CPI isoladamente.

3.22 Comparando P-core e E-core

Uma CPU híbrida permite um experimento particularmente interessante.

Primeiro identifique as CPUs:

```
cat /sys/bus/event_source/devices/cpu_core/cpus
cat /sys/bus/event_source/devices/cpu_atom/cpus
```

Depois rode o mesmo executável em um P-core e em um E-core, mantendo todas as demais condições iguais.

Uma tabela de resultados pode conter:

Métrica	P-core	E-core
Tempo [s]	—	—
Cycles	—	—
Instructions	—	—
IPC	—	—
Branch miss rate	—	—
Cache miss rate	—	—
Energia [J]	—	—
Potência média [W]	—	—
EDP	—	—

Esse experimento permite discutir desempenho absoluto, eficiência energética e heterogeneidade dentro de um único processador.

3.23 Cuidados experimentais em HPC

Uma medição tecnicamente correta depende de mais do que executar `perf stat`. Recomenda-se:

1. usar a mesma versão do executável e os mesmos dados de entrada;
2. registrar compilador e flags de compilação;
3. registrar CPU, kernel e versão do `perf`;
4. fixar afinidade quando a migração pode interferir;
5. evitar carga concorrente desnecessária;
6. repetir cada teste várias vezes;
7. controlar ou registrar políticas de frequência da CPU;
8. considerar aquecimento e throttling em execuções longas;
9. não comparar diretamente contadores coletados com alto grau de multiplexação sem analisar o tempo ativo;
10. confirmar que os eventos selecionados possuem o mesmo significado nas arquiteturas comparadas;
11. usar tamanhos de problema suficientemente grandes para reduzir o peso do overhead de inicialização;
12. validar numericamente a saída antes de comparar desempenho.

3.24 `perf` versus PAPI e LIKWID

As três ferramentas podem trabalhar com contadores de desempenho, mas possuem filosofias diferentes.

	`perf`	PAPI	LIKWID
Camada principal	ferramenta do ecossistema Linux	API portável	ferramentas + API/Marker
Integração com kernel	direta via `perf_events`	usa backends da plataforma	acessos/abstrações próprias e do SO
Instrumentação no código	opcional	típica	opcional/Marker API
Sampling por função	sim	não é o foco principal	limitado em comparação ao `perf`
Top-down	sim, quando suportado	via eventos/métricas montadas	grupos específicos
Eventos de software/kernel	forte	não é o foco	não é o foco
Portabilidade entre SOs	Linux	maior	principalmente Linux/HPC

Para ensino de HPC, as ferramentas são complementares:

- PAPI é excelente para instrumentar trechos diretamente no programa;
- LIKWID fornece grupos de métricas e afinidade de forma didática;
- `perf` oferece integração profunda com Linux, PMUs, scheduler, tracing e sampling.

3.25 Comandos de referência rápida

Comando	Finalidade
`perf --version`	Mostra a versão instalada.
`perf list`	Lista todos os eventos conhecidos na máquina.
`perf list hw`	Eventos genéricos de hardware.
`perf list sw`	Eventos de software.
`perf list cache`	Eventos de cache genéricos.
`perf list pmu`	Eventos associados às PMUs.
`perf list metric`	Métricas derivadas disponíveis.
`perf stat ./prog`	Estatísticas gerais da execução.
`perf stat -e EVENTO ./prog`	Mede evento específico.
`perf stat -d ./prog`	Acrescenta estatísticas detalhadas.
`perf stat -r 10 ./prog`	Repete a execução 10 vezes.
`perf stat --topdown ./prog`	Solicita análise Top-down, quando suportada.
`perf record ./prog`	Coleta amostras e grava `perf.data`.
`perf report`	Exibe o perfil gravado.
`perf annotate`	Relaciona amostras com código/assembly.
`taskset -c N ./prog`	Fixa o programa na CPU lógica N.

3.26 Conclusão

O `perf` permite passar de uma avaliação superficial baseada apenas em tempo para uma análise baseada na dinâmica real da microarquitetura e do sistema operacional. Em um problema de HPC, a sequência de perguntas pode ser formulada como:

O programa está correto?
↓
Quanto tempo leva?
↓
Quantos ciclos e instruções executa?
↓
Qual é o IPC/CPI?
↓
Há problemas de branch prediction?
↓
O gargalo é frontend, backend ou especulação?
↓
Se é backend, trata-se de memória/cache ou recursos do core?
↓
Quais funções concentram o custo?
↓
A otimização reduziu tempo e/ou energia?

Essa abordagem transforma o profiling em um processo científico: formular uma hipótese sobre o gargalo, selecionar eventos capazes de testá-la, controlar as condições experimentais, coletar dados, interpretar os resultados e repetir o experimento após cada transformação do código.